

AK VisionFlow

PHASE 0.3 — MULTI-TENANT / COMPANY / BRANCH ARCHITECTURE

Short Developer Implementation Specification

Objective: Establish a clean Tenant → Company → Branch architecture with secure data isolation and controlled cross-branch access.

1. Scope

- Tenant/company hierarchy, branches, warehouses, users, roles, permissions, branch access, database relationships, API enforcement, migration and security testing.
- Do not redesign Accounting, AI, Owner Mobile or later ERP modules.

2. Target Hierarchy

- **Tenant:** isolated customer/business account.
- **Company:** business/legal operating entity.
- **Branch:** operational location belonging to a company.
- **Warehouse:** stock location associated with the relevant branch/company.
- Users may access one or multiple branches according to permissions.

3. Data Ownership

- Every tenant-owned record must be tenant-scoped.
- Branch-specific transactions must carry and enforce branch scope.
- Company-level master data may be shared only when explicitly configured.
- Tenant membership alone must never imply access to every branch.

4. Access Control

- Centralize tenant and branch authorization in reusable middleware/service helpers.
- Permissions must define both what a user can do and where they can do it.
- Support own-branch, selected-branch and company-wide access where authorized.
- Cross-tenant access must always fail.

5. Current Architecture Refactoring

- Standardize inconsistent branch-scope implementation into one common convention.
- Address documented gaps affecting warehouses, stock transfers, procurement, journal/report routes and other branch-sensitive operations.
- Do not change working business behavior except where required for correct scope enforcement.

6. Database & API

- Define Tenant → Company → Branch relationships with appropriate foreign keys and indexes.
- Every tenant-owned API must enforce tenant scope server-side.
- Branch-sensitive APIs must enforce branch authorization server-side.
- Never trust a client-supplied branchId without validating access.

7. Frontend

- Branch selectors show only authorized branches.
- UI filtering is convenience only; backend authorization remains authoritative.

- Company/branch context must remain consistent across dashboards, transactions and reports.
- Do not expose inaccessible branches through URLs or cached data.

8. Migration

- Inventory existing records and determine ownership.
- Map legacy records before migration.
- Backfill safely in a test environment first.
- Maintain rollback/recovery procedures and run regression after migration.

9. Security & Testing

- Cross-tenant IDOR tests.
- Unauthorized cross-branch read/write tests.
- Authorized multi-branch access tests.
- Warehouse/stock-transfer, procurement, journal and report scope tests.
- User/role/branch assignment and migration integrity tests.

10. Acceptance Criteria

- No confirmed cross-tenant data access.
- Branch-sensitive APIs consistently enforce authorization.
- Users access only permitted branch data.
- Authorized company-wide operations remain available.
- Existing Optical/Medical workflows continue working.
- Automated tests prove tenant and branch isolation.

11. Phase Gate

- Document the final hierarchy, permission model, migration results and security/regression evidence.
- Product Owner approval is required before proceeding to the next phase.

Developer Checklist

Area	Check	Status
Hierarchy	Tenant → Company → Branch model	■
Authorization	Central tenant/branch enforcement	■
Database	FKs, indexes and constraints	■
APIs	Branch-sensitive routes protected	■
Frontend	Branch context respects access	■
Migration	Legacy mapping/backfill verified	■
Security	Cross-tenant/cross-branch tests pass	■
Regression	Optical/Medical tests pass	■
Approval	Product Owner phase gate	■

Definition of Done: Tenant, company and branch boundaries are explicit, consistently enforced server-side, tested for isolation, and compatible with existing business workflows.